AI/ML Engineer Assessment: Multimodal Submission Integrity & Plagiarism Detection Engine

1. Overview

This assessment evaluates your ability to design and implement an end-to-end multimodal content verification and similarity detection pipeline. You will demonstrate core machine learning competencies: text extraction across formats, vector indexing and similarity search, multi-factor integrity scoring (peer-to-peer, web reference, and AI-generated text estimation), and explainable cross-referencing. This project is tailored for candidates entering an AI/ML Engineer role.

Role / Level	Duration	Format	Tools Allowed	Submission Method
AI/ML Engineer (Fresher / Junior)	72 hours (Est. 8–10 active hours)	Python Project (Code + Evaluation + Architecture Brief)	Python 3.9+, Open-source ML libraries, Hugging Face, FAISS/Chroma, OpenAI Whisper (or equivalent open models)	GitHub Repository link via submission form

2. Background and Problem Statement

Our platform manages large-scale case competitions and university evaluations, receiving upwards of 10,000 submissions per challenge. These submissions arrive in multiple formats: PDF reports, DOCX executive summaries, and MP4 video pitches.

To maintain academic and competitive fairness, evaluation teams need to identify non-original work quickly and accurately. Traditional word-matching tools fail because students submit content across different modalities (for example, reading an online article verbatim in an MP4 video or paraphrasing a peer's PDF submission). Furthermore, a bare percentage score is not actionable; reviewers need precise references, highlighted

text overlaps, and human-verifiable citations to justify any penalty. Your challenge is to build a working prototype that extracts, indexes, matches, and explains submission overlap across modalities.

3. Your Objective

Build a functional Python pipeline that ingests a sample corpus of multi-format submissions (PDF, DOCX, MP4), normalizes and indexes their textual representations, calculates distinct similarity scores (internal peer-to-peer similarity, external/web reference similarity, and AI-generated probability), and generates an explainable audit report containing exact reference matches for cross-verification.

4. What You Need to Do

Task 1: Multimodal Ingestion and Normalization (Weight: 20%)

- Build a document parsing utility that extracts clean text from `.pdf`, `.docx`, and `.mp4` (extract audio track and transcribe to text using an open model like Whisper-base/tiny or Vosk).
- Clean and normalize extracted text: handle whitespace, strip headers/footers, and segment the text into meaningful chunks (e.g., 200–400 tokens with slight overlap) while preserving source metadata (file name, page number, timestamp in video).
- Handle malformed or unreadable files gracefully without crashing the pipeline.

Task 2: Indexing and Internal Peer-to-Peer Similarity (Weight: 25%)

- Convert chunked submissions into dense vector embeddings using a lightweight, open-source embedding model (e.g., `all-MiniLM-L6-v2` or similar from Sentence-Transformers).
- Index these embeddings using an in-memory or file-backed vector index (e.g., FAISS, ChromaDB, or NumPy cosine similarity matrix).
- Implement a matching algorithm that compares each document against all other documents in the corpus to produce an internal peer similarity percentage. Ensure identical self-matches are excluded.

Task 3: Multi-Signal Integrity Detection (External & AI Signals) (Weight: 20%)

- **AI-Generation Estimation:** Implement an AI-content indicator using either an open-source classifier (e.g., a Hugging Face RoBERTa-based AI detector), perplexity/burstiness heuristics, or a documented mock wrapper with reproducible logic.
- **Web/External Plagiarism Engine:** Implement an external similarity mechanism. Since full-scale web scraping can hit rate limits and costs, you may simulate external reference search using a local reference knowledge base (e.g., a set of Wikipedia articles or case study source texts) or a free search API with fallback mocks.
- Combine these signals into a structured breakdown: Internal Peer Similarity %, External Match %, and AI Content Likelihood %.

Task 4: Citation and Explainability Engine (Weight: 20%)

- For every detected overlap, output the exact matching passages:
  - The offending sentence/chunk from the query document.
  - The source sentence/chunk from the matching document.
  - Metadata: matching file name, page number or video timestamp, and chunk similarity score.
- Ensure the output report gives human reviewers sufficient context to cross-verify claims in under 30 seconds.

Task 5: 10,000 Document Scaling Brief (Weight: 15%)

- Write a short section in your README explaining how your prototype would scale to process 10,000 submissions concurrently in production:
  - Compute choices (GPU/CPU) and asynchronous processing queues.
  - Vector database selection and storage indexing strategies (e.g., HNSW vs. Flat).
  - Cost, throughput, and latency bottlenecks.

Out of Scope

- Building a complex frontend UI (a CLI or clean Python script outputting JSON/Markdown reports is preferred).
- Subscribing to paid, commercial plagiarism APIs (Turnitin, Copyleaks, etc.).
- Transcribing hours of video—keep sample MP4 clips short (10–60 seconds).

5. Scenarios to Handle

Your pipeline will be evaluated against the following scenarios:

Scenario Name	Situation	Input	Expected Behavior or Outcome
Scenario 1: Cross-Modal Duplication	A student reads a paragraph from another student's PDF submission in their MP4 video pitch.	1 PDF report and 1 short MP4 file with identical spoken text.	Pipeline extracts transcript from MP4, matches it against the PDF vector index, and flags the overlapping segment with video timestamp and PDF page reference.
Scenario 2: Paraphrased Content	Content has been reworded using synonyms to evade naive keyword matching.	Two DOCX files expressing the same distinct arguments with altered phrasing.	Semantic embeddings flag a high similarity match (e.g., cosine similarity > 0.80) even when exact string-matching metrics (e.g., Jaccard/Levenshtein) are low.
Scenario 3: Corrupted / Empty Submissions	A corrupted PDF or an MP4 file with no audio track is submitted.	1 zero-byte PDF and 1 silent MP4 video.	Pipeline logs an informative warning, skips or flags the file without crashing, and processes all remaining documents.
Scenario 4: Benign Common Formatting (Judgment Test)	Multiple case studies quote the exact same case prompt, question instructions, and citations.	5 documents containing identical 150-word problem prompt headers, but completely original analytical answers.	Pipeline does not mark the entire submission as heavily plagiarized; it correctly isolates the common template or discounts short, repeated institutional boilerplate.
Scenario 5: Multi-Document Evaluation (Stress Test)	A batch of mixed files is submitted in a single run.	A folder of 10–20 test documents containing a mix of genuine, copied, and AI-	Pipeline processes the entire directory, generating an aggregate summary matrix and individual audit reports for each file.

		generated samples.	

Note: Additional hidden edge-case submissions will be tested during evaluation.

6. Expected Output

You must deliver a clean, modular Python codebase accompanied by documentation and test results.

Deliverables

1. **Source Code:** Modular Python project containing extractors, indexer, similarity logic, and reporter.
2. **CLI or Runner Script:** A script that accepts an input folder containing candidate files and writes out verification reports:

   ```
   python run_integrity_check.py --input_dir ./sample_submissions --output_dir ./rep
   ```

3. **Structured Reports:** For each document evaluated, output a `.json` report and a human-readable `.md` report.
4. **README.md:** Documentation covering setup, methodology, evaluations, and scaling architecture.

Output Format Standard (JSON Schema Example)

Each generated report must follow this structure:

```
{
  "document_id": "submission_042.pdf",
  "total_integrity_risk": "HIGH",
  "scores": {
    "peer_similarity_percentage": 78.5,
    "external_similarity_percentage": 12.0,
    "ai_generated_probability": 85.0
  },
  "peer_matches": [
    {
      "matched_file": "submission_009.docx",
      "similarity_score": 0.91,
```

```
      "query_passage": "The core competitive advantage relies on vertical integration of su
      "matched_passage": "Their core competitive advantage is built upon vertical integrat
      "location": {
        "query_page": 2,
        "matched_page": 1
      }
    }
  ],
  "multimodal_matches": [
    {
      "matched_file": "pitch_015.mp4",
      "similarity_score": 0.88,
      "query_passage": "Operating margins contracted by 4.2% over Q3 due to supply cha
      "matched_passage": "Operating margins dropped 4.2% across the third quarter beca
      "location": {
        "query_page": 3,
        "matched_timestamp": "00:00:24 - 00:00:36"
      }
    }
  ],
  "explanations": "High internal peer similarity detected across Section 2 and Section 4. H
}
```


7. Sample Data and Files

No pre-packaged dataset is provided. **You must construct a minimal, diverse test dataset of 10 to 15 sample submissions** to test and demonstrate your engine.

Dataset Construction Guidelines

- **File Types:** Include at least 5 PDFs, 5 DOCX files, and 2–3 short MP4 clips (10–30 seconds each, containing spoken English).
- **Embedded Ground Truth:** You must intentionally craft relationships between these files:
    - At least 1 pair of cross-document exact copies.
    - At least 1 pair of cross-modal matches (e.g., an MP4 speech matching a section of a PDF).
    - At least 1 document generated via an LLM (e.g., ChatGPT/Claude) to test AI detection.

    - At least 2 completely original, independent submissions to test false positives.
    - At least 1 degraded file (e.g., silent MP4, empty file, or heavily formatted PDF).
- **Data Compliance:** Do not use proprietary, copyrighted, or private student data. Use synthetic case studies, public Wikipedia summaries, or open domain texts.

8. What You Must Submit With Your Solution

Ensure the following items are committed to your repository:

1. **Sample Data Directory (`/sample_submissions`):** The exact multi-format files you used to evaluate your system, accompanied by a `dataset_manifest.json` describing the ground truth (what is plagiarized from what).
2. **Supporting Files:** `requirements.txt` or `pyproject.toml`, model download scripts (if applicable), and configuration files.
3. **Your Own Evaluations (`EVALUATION.md`):**
    - **Metrics:** Report Precision, Recall, or Confusion Matrix against your synthetic ground truth.
    - **Failure Analysis:** Detail at least 2 instances where your prototype failed or produced ambiguous results (e.g., false positive on common jargon, transcription errors on accented speech), and explain how you would resolve them.
    - **Trade-off Analysis:** Explain your choice of embedding model and vector index versus compute constraints.
4. **Scaling Architecture Note (in `README.md`):** Your concrete plan for taking this system from 15 files to 10,000 files in production.

Submission Checklist

- ☐ Working ingestion pipeline for PDF, DOCX, and MP4 files.
- ☐ Vector similarity search pipeline with exclusion of self-matches.
- ☐ AI-generation and external similarity estimation modules.
- ☐ Human-readable explainability report with exact chunk matches and timestamps/page numbers.
- ☐ Synthetic dataset (10–15 files) with ground-truth manifest included in repo.
- ☐ `EVALUATION.md` with performance metrics and failure case analysis.
- ☐ Scaling brief detailing handling of 10,000 files.
- ☐ Clean `README.md` with single-command setup instructions.

9. Evaluation Criteria

Your submission will be scored based on the following rubric:

Criterion	What We Look For	Weight (%)
Correctness & Core Functionality	Clean extraction from PDF/DOCX/MP4, robust vector indexing, correct mathematical computation of similarity, and handling self-match exclusions.	25%
Handling of Scenarios & Multimodal Edge Cases	Graceful handling of silent video, empty files, template boilerplate, and cross-modal matches (MP4 transcript to PDF text).	20%
Quality & Rigor of Own Evaluation	Systematic evaluation against ground truth, inclusion of confusion metrics/tables, honest evaluation of false positives/negatives, and self-awareness of limitations.	20%
Explainability & Report Quality	Detailed, structured output matching snippets, timestamps, page references, and clear human-readable explanations instead of opaque scores.	15%
Architecture & Scaling Reasoning	Practicality, depth, and technical feasibility of the architectural design for processing 10,000 submissions.	10%
Code Quality & Documentation	Clean, modular Python code (PEP 8), clear documentation, reproducible environment setup, and thoughtful error handling.	10%

Performance Tiers

- **Exceptional:** Flawless cross-modal matching, robust chunking with page/timestamp offsets, clear ground-truth benchmark with high recall/precision, and a production-ready scaling architecture.
- **Strong:** Clean ingestion across all 3 formats, working vector similarity, transparent explanation outputs, and realistic evaluation metrics. Minor edge-case misses.
- **Meets Expectations:** Basic ingestion works, internal text similarity produces percentages, some explainability provided, but struggles with video audio

extraction or edge cases.
- **Unsatisfactory:** Fails to handle multi-format files, code crashes on malformed inputs, opaque percentage outputs with no matched references, or absence of self-evaluation.

10. Rules and Guidelines

- **Time Limit:** 72 hours from receipt of this brief.
- **AI Coding Assistants:** You may use AI coding assistants (e.g., GitHub Copilot, ChatGPT) for boilerplate and debugging, but you must disclose your usage in your README. You must be prepared to explain all architectural decisions and code lines during a follow-up technical interview.
- **Dependencies:** Stick to open-source libraries that run on standard commodity hardware (CPU or free-tier Colab GPU). If a model requires large downloads, use smaller variants (e.g., `openai/whisper-tiny` or `whisper-base`, `sentence-transformers/all-MiniLM-L6-v2`).
- **Assumptions:** If any requirement seems ambiguous, make a reasoned assumption, document it under an "Assumptions" heading in your `README.md`, and proceed.

11. Submission Instructions

- Push your code, test dataset, and documentation to a private GitHub repository.
- Grant access to the evaluation handle provided in your invitation email (or make it public if comfortable).
- Submit the repository URL and commit hash via the candidate portal before the 72-hour deadline.
- Ensure your repository contains a single root command to run the pipeline over your sample data.
- For technical blockers or submission link issues, contact `engineering-hiring@company.com`.

12. FAQs

Q: Am I expected to transcribe hours of high-definition video?
A: No. Keep sample MP4 video files between 10 to 60 seconds. We are evaluating your pipeline integration (extracting audio, running an ASR model, and chunking the transcript into the vector index), not your GPU rendering capacity.

Q: What if I don’t have access to paid AI-detection APIs?
A: Do not pay for APIs. Use open-source Hugging Face classification models, statistical heuristics (perplexity/burstiness), or a structured wrapper that demonstrates the analytical integration. Explain your chosen approach in your README.

Q: Can I use LangChain, LlamaIndex, or raw Python?
A: You may use frameworks like LangChain or LlamaIndex, or build directly with Sentence-Transformers and FAISS. We prioritize candidates who understand the underlying mechanics (chunking, embeddings, dot-products/cosine distance, thresholding) rather than those who blindly chain high-level wrappers.

Q: Do I need to build a web application interface?
A: No. A CLI script that takes an input folder and outputs Markdown/JSON files is the expected deliverable. Spend your time on the ML pipeline, explainability references, and edge cases.